REST ASSURED

Class 7: Nested POJOs and JSON Serialization

Beginner-friendly complete notes with explained code, real-world testing scenarios, best practices, and hands-on exercises

Core idea Model the shape of a JSON request as Java classes, connect nested objects, and let Jackson/Rest Assured serialize the object graph into JSON.

Based on the uploaded “Rest Assured class 7.txt” material: nested user/address/company payloads, nested product/category payloads, lists of images, and users with skills and multiple avatar objects.

Learning target: be able to read a JSON contract, design matching POJOs, populate nested objects and collections, send the request, and assert the API response.

Important: public practice APIs and their behavior can change. Treat URLs from the source notes as examples; confirm the current API contract and response code before relying on them.

Contents and learning roadmap

Section	What you will learn
1. Foundations	JSON, POJO, object graph, serialization and deserialization
2. Project setup	Maven dependencies, Jackson, Lombok and Rest Assured
3. Nested user model	Users → Address → Geo and Company
4. Serialization choices	request.body(pojo) vs ObjectMapper.writeValueAsString
5. Nested product payload	Products → Category + List<String> images
6. Collections of nested objects	List<String> skills and List<Avatar>
7. End-to-end tests	TestNG tests, assertions, response checks and examples
8. Debugging	Common source-code issues and how to fix them
9. Best practices	DTO design, types, secrets, logs, maintainability
10. Hands-on	Exercises, solutions and interview revision

Suggested study order

1. Learn what a POJO is and how a Java object maps to a JSON object.
2. Create simple classes with primitive/string fields and getters/setters or Lombok.
3. Create nested classes from the inside out: Geo, then Address, then Users.
4. Add collections such as images, skills and avatar objects.
5. Serialize, send the request and verify meaningful response fields.
6. Complete exercises without copying the solution first.

1. Start with the JSON shape

What is JSON?

JSON (JavaScript Object Notation) is a text format commonly used for API request and response bodies. A JSON object uses key/value pairs. Nested objects are objects inside other objects; arrays hold ordered lists of values or objects.

```
{
 "name": "Mayuri",
 "email": "mayuri@example.test",
 "address": {
  "city": "Pune",
  "geo": {
   "lat": "24.455",
   "lng": "73.455"
  }
 },
 "company": {
  "name": "Amdocs",
  "catchPhrase": "Testing Phase"
 }
}
```

JSON concept	Example	Java model
String field	"name": "Mayuri"	private String name;
Nested object	"address": { ... }	private Address address;
Array of strings	"images": ["a.png", "b.png"]	private List<String> images;
Array of objects	"avatar": [{...}, {...}]	private List<Avatar> avatar;
Boolean	"status": true	private Boolean status;
Integer	"size": 40000	private Integer size;

Beginner rule Do not design a nested POJO by guessing. Read the API response/request example or Swagger/OpenAPI definition first, then mirror the field names, nesting and types.

2. What is a POJO, and why use it?

POJO stands for Plain Old Java Object. In API automation, a POJO is a normal Java class used to represent request or response data. Instead of manually assembling a large JSON string, you create Java objects and set their values.

Without a POJO

```
String body = "{\"name\":\"Mayuri\",\"email\":\"mayuri@example.test\"}";
request.body(body);
```

This is short only for a very small body. For nested structures, manual JSON strings become difficult to read and easy to break with commas, quotes or escaped characters.

With a POJO

```
Users user = new Users();
user.setName("Mayuri");
user.setEmail("mayuri@example.test");
request.body(user);
```

The class describes the fields; the object holds the values for one request. Jackson converts the Java object to JSON when configured as the object mapper used by Rest Assured.

Term	Meaning
Class	Blueprint defining fields and behavior, e.g. Users.
Object / instance	One created value, e.g. a user named Mayuri.
Field	A data member, e.g. private String email.
Getter	Method that returns a field value.
Setter	Method that assigns a field value.
Serialization	Converting a Java object into JSON text.
Deserialization	Converting JSON into a Java object.

3. Dependencies and project setup

Use dependency versions that are compatible with the Java version and the project. This is a representative Maven setup; if your existing project already manages versions through a parent POM or BOM, keep the project's version policy consistent.

```
<dependencies>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>YOUR_PROJECT_VERSION</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>YOUR_PROJECT_VERSION</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>YOUR_PROJECT_VERSION</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>YOUR_PROJECT_VERSION</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Replace YOUR_PROJECT_VERSION with versions approved for your project; the placeholders above are intentional rather than copy-paste versions.

What each dependency does

Dependency	Purpose
Rest Assured	Build HTTP requests and inspect responses.
Jackson Databind	Maps Java objects to JSON and JSON to Java objects.
TestNG	Runs tests and provides assertions/lifecycle annotations.
Lombok	Generates boilerplate methods such as getters/setters during compilation.

Lombok setup Your IDE/build must support Lombok annotation processing. If the code compiles in Maven but the IDE shows errors—or vice versa—check the Lombok plugin and annotation-processing configuration.

4. Lombok and Java fields

Manual getters and setters

```
public class Geo {
    private String lat;
    private String lng;

    public String getLat() {
        return lat;
    }

    public void setLat(String lat) {
        this.lat = lat;
    }

    public String getLng() {
        return lng;
    }

    public void setLng(String lng) {
        this.lng = lng;
    }
}
```

private means other classes cannot directly access the fields. A setter provides a controlled way to assign a value; a getter returns the value. The expression this.lat refers to the current object's field, while lat on the right side is the method parameter.

Using Lombok @Data

```
import lombok.Data;

@Data
public class Geo {
    private String lat;
    private String lng;
}
```

@Data generates getters, setters, toString(), equals() and hashCode() for the class. It reduces repetitive code. It does not decide whether fields or business values are correct; validation and API assertions are still your responsibility.

Caution with @Data For classes containing passwords, tokens or other secrets, generated toString() can expose values in logs. Prefer narrower annotations or explicitly exclude sensitive fields from toString().

5. Build the nested Users payload

The source material models a JSONPlaceholder-style user with address and company objects. The object structure is: Users contains Address and Company; Address contains Geo. Build the leaf objects first, then place them into their parent objects.

```
// Users.java
import lombok.Data;

@Data
public class Users {
    private String name;
    private String username;
    private String email;
    private Address address;
    private String phone;
    private String website;
    private Company company;
}

// Address.java
import lombok.Data;

@Data
public class Address {
    private String street;
    private String suite;
    private String city;
    private String zipcode;
    private Geo geo;
}

// Geo.java
import lombok.Data;

@Data
public class Geo {
    private String lat;
    private String lng;
}

// Company.java
import lombok.Data;

@Data
public class Company {
    private String name;
    private String catchPhrase;
    private String bs;
}
```

How should you read these fields?

Field	Why this type?	What JSON it represents
String name	Text field	"name": "Mayuri"
Address address	Address is another object	"address": { ... }
Geo geo	Geo is nested inside Address	"geo": { "lat": ..., "lng": ... }
Company company	Company is a separate nested object	"company": { ... }

Construct the object graph from the inside out

```
Company company = new Company();
company.setName("Amdocs");
company.setCatchPhrase("Testing Phase");
company.setBs("Telecom Domain");

Geo geo = new Geo();
geo.setLat("24.455");
geo.setLng("73.455");

Address address = new Address();
address.setStreet("GM Road");
address.setSuite("A-290");
address.setCity("Pune");
address.setZipcode("112233");
address.setGeo(geo);

Users user = new Users();
user.setName("Mayuri");
user.setUsername("mayuri");
user.setEmail("mayuri@example.test");
user.setAddress(address);
user.setPhone("9090901122");
user.setWebsite("https://example.test");
user.setCompany(company);
```

When user.setAddress(address) runs, the user object stores a reference to the already-built Address object. That address in turn refers to Geo. Serialization follows these relationships and creates nested JSON.

Important data quality point In the original notes, lng is set to "25.45.45", which is not a valid ordinary decimal coordinate. Use a valid value in a realistic test and assert the business/API rules that apply.

6. Serialization: two practical ways

Option A — Pass the POJO directly to Rest Assured

```
RequestSpecification request = RestAssured.given();
request.contentType("application/json");
request.body(user);

Response response = request.post("https://jsonplaceholder.typicode.com/users");
response.then().log().ifValidationFails();
```

This is the simplest approach. Rest Assured uses its configured object-mapping support to serialize user into JSON. Ensure Jackson Databind is on the test classpath and the correct content type is set.

Option B — Serialize explicitly with ObjectMapper

```
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;

ObjectMapper mapper = new ObjectMapper();
String jsonUserRequestBody = mapper.writeValueAsString(user);

request.contentType("application/json");
request.body(jsonUserRequestBody);
Response response = request.post("https://jsonplaceholder.typicode.com/users");
```

ObjectMapper.writeValueAsString converts the Java object graph into JSON text. The method can throw JsonProcessingException, so the test must handle or declare that exception. Explicit serialization is useful when you need to inspect or transform the JSON before sending it.

Approach	When to choose it	Common check
request.body(user)	Preferred for simple POJO request bodies	Jackson available; correct content type; expected field names
ObjectMapper + String	When the JSON string must be inspected, stored or customized	Catch/declare JsonProcessingException; ensure mapper config matches the API

7. Full beginner-friendly TestNG example

The original source often uses main(). That is useful for a quick demo. In an automation framework, prefer TestNG @Test methods so failures appear clearly in test reports and can run in CI.

```
package tutorial1;

import static org.hamcrest.Matchers.*;
import org.testng.annotations.Test;
import io.restassured.RestAssured;
import io.restassured.response.Response;

public class UsersApiTest {

    @Test
    public void createUserWithNestedObjects() {
        Company company = new Company();
        company.setName("Amdocs");
        company.setCatchPhrase("Testing Phase");
        company.setBs("Telecom Domain");

        Geo geo = new Geo();
        geo.setLat("24.455");
        geo.setLng("73.455");

        Address address = new Address();
        address.setStreet("GM Road");
        address.setSuite("A-290");
        address.setCity("Pune");
        address.setZipcode("112233");
        address.setGeo(geo);

        Users user = new Users();
        user.setName("Mayuri");
        user.setUsername("mayuri");
        user.setEmail("mayuri@example.test");
        user.setAddress(address);
        user.setPhone("9090901122");
        user.setWebsite("https://example.test");
        user.setCompany(company);

        Response response = RestAssured.given()
            .contentType("application/json")
            .body(user)
            .post("https://jsonplaceholder.typicode.com/users");

        response.then()
            .statusCode(anyOf(is(200), is(201)))
            .contentType(containsString("application/json"))
            .body("name", equalTo("Mayuri"))
            .body("address.city", equalTo("Pune"))
            .body("company.name", equalTo("Amdocs"));
    }
}
```

Expected status caveat The anyOf(200, 201) assertion is only an illustration for a mock/demo endpoint whose behavior you have not pinned down. In a real test, agree the exact expected status with the API contract and assert that exact value.

How to understand the assertions

Code	Meaning
statusCode(anyOf(is(200), is(201)))	For this sample only, accept either documented demo outcome; real tests should be stricter.
contentType(containsString("application/json"))	Check that the response is JSON; exact media-type behavior depends on the API.

Code	Meaning
body("name", equalTo("Mayuri"))	Read the root-level JSON name field.
body("address.city", equalTo("Pune"))	Use dot notation to read a nested field.
body("company.name", equalTo("Amdocs"))	Read a field nested two levels under company.

8. Example 2 — Product with nested Category and image list

The source product model includes a nested Category object and a list of image URLs. The JSON should be shaped like this:

```
{
  "title": "Samsung 10% Offer",
  "slug": "samsung-offer",
  "price": 23999,
  "description": "Offers on selected mobile phones",
  "category": {
    "id": 1,
    "name": "Electronics",
    "slug": "electronics",
    "image": "https://example.test/category.jpg",
    "creationAt": "2026-09-28T17:21:29.000Z",
    "updatedAt": "2026-09-28T17:21:29.000Z"
  },
  "images": [
    "https://example.test/a1.png",
    "https://example.test/a2.png",
    "https://example.test/a3.png"
  ]
}
```

```
// Category.java
@Data
public class Category {
    private Integer id;
    private String name;
    private String slug;
    private String image;
    private String creationAt;
    private String updatedAt;
}

// Products.java
import java.util.List;
import lombok.Data;

@Data
public class Products {
    private String title;
    private String slug;
    private Integer price;
    private String description;
    private Category category;
    private List<String> images;
    private String creationAt;
    private String updatedAt;
}
```

Construct and send the product

```
Category category = new Category();
category.setId(1);
category.setName("Electronics");
category.setSlug("electronics");
category.setImage("https://example.test/category.jpg");
category.setCreationAt("2026-09-28T17:21:29.000Z");
category.setUpdatedAt("2026-09-28T17:21:29.000Z");

Products product = new Products();
product.setTitle("Samsung 10% Offer");
product.setSlug("samsung-offer");
product.setPrice(23999);
product.setDescription("Offers on selected mobile phones");
product.setCategory(category);
product.setImages(List.of(
   "https://example.test/a1.png",
   "https://example.test/a2.png",
   "https://example.test/a3.png"));
product.setCreationAt("2026-09-28T17:21:29.000Z");
product.setUpdatedAt("2026-09-28T17:21:29.000Z");

Response response = RestAssured.given()
   .contentType("application/json")
   .body(product)
   .post("https://api.escuelajs.co/api/v1/products");

response.then().log().ifValidationFails();
```

List.of is available in Java 9 and newer. It returns an unmodifiable list. If your test needs to add/remove images after creation, use an ArrayList instead.

Real-world scenario: e-commerce catalogue

Scenario: A catalogue service creates a product with a category and several product images. Test both the parent and child data, not just the top-level title.

```
response.then()
   .statusCode(201) // use the exact status from the API contract
   .body("title", equalTo("Samsung 10% Offer"))
   .body("price", equalTo(23999))
   .body("category.name", equalTo("Electronics"))
   .body("images.size()", equalTo(3))
   .body("images", hasItem("https://example.test/a2.png"));
```

Contract-first modelling Some APIs require only category ID in a create request and reject an entire category object. The source example provides a nested category object, but the request must match the target API contract. Do not assume a response model is automatically a valid request model.

9. Example 3 — User with skills and multiple avatars

This example has two collection types: a list of strings for skills and a list of nested Avatar objects. A Java List<Avatar> becomes a JSON array of objects.

```
// Users.java
@Data
public class Users {
    private String firstName;
    private String lastName;
    private String email;
    private Boolean status;
    private String birthday;
    private List<String> skills;
    private List<Avatar> avatar;
}

// Avatar.java
@Data
public class Avatar {
    private String name;
    private Integer percent;
    private Integer size;
    private String status;
    private String type;
    private String uid;
    private String url;
}
```

Create the collection and parent object

```
Avatar avatar1 = new Avatar();
avatar1.setName("abc1.png");
avatar1.setPercent(100);
avatar1.setSize(40000);
avatar1.setStatus("done");
avatar1.setType("image/png");
avatar1.setUid("abc-abc-1333");
avatar1.setUrl("https://example.test/abc1.png");

Avatar avatar2 = new Avatar();
avatar2.setName("xyz1.png");
avatar2.setPercent(75);
avatar2.setSize(50000);
avatar2.setStatus("done");
avatar2.setType("image/png");
avatar2.setUid("abc-xyz-1333");
avatar2.setUrl("https://example.test/xyz1.png");

List<Avatar> avatars = List.of(avatar1, avatar2);
List<String> skills = List.of("Playwright", "Selenium", "Rest Assured", "Docker");

Users user = new Users();
user.setFirstName("Soham");
user.setLastName("Shinde");
user.setEmail("soham@example.test");
user.setStatus(true);
user.setBirthday("2025-10-21T20:45:47.563Z");
user.setSkills(skills);
user.setAvatar(avatars);
```

Notice that the variable type is List<Avatar>, not List<Avater> or List<Avatars>. The generic type must match the class name exactly.

Real-world scenario: profile onboarding

A profile page allows a user to select multiple skills and upload multiple avatar images. Useful checks include the Boolean status, non-empty skills, number of avatars, and fields of each avatar.

```
response.then()
   .body("status", equalTo(true))
   .body("skills", hasItems("Selenium", "Docker"))
   .body("avatar.size()", equalTo(2))
   .body("avatar[0].name", equalTo("abc1.png"))
   .body("avatar[1].type", equalTo("image/png"));
```

10. How JSON and nested POJOs fit together

Think of a parent POJO as a box that contains references to other boxes. Serialization walks the graph and writes nested objects and lists as JSON structures.

```
Users
├── name: String
├── address: Address
│   ├── street: String
│   └── geo: Geo
│       ├── lat: String
│       └── lng: String
└── company: Company
    ├── name: String
    ├── catchPhrase: String
    └── bs: String
```

Java field declaration	JSON result shape
private Address address;	"address": { ... }
private Geo geo;	"geo": { "lat": "...", "lng": "..." }
private List<String> images;	"images": ["...", "..."]
private List<Avatar> avatar;	"avatar": [{...}, {...}]

11. JSON field names that do not match Java naming

Sometimes JSON keys are awkward or reserved-word-like. The source file shows JSON field "long" represented with a Java field geoLong. Jackson can map them explicitly with @JsonProperty.

```
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Data;

@Data
public class Geolocation {
   private String lat;

   @JsonProperty("long")
   private String geoLong;
}
```

When serialized, geoLong maps to the JSON property "long" because of @JsonProperty("long"). Be careful with global Jackson naming settings: a custom ObjectMapper configuration can affect how fields are serialized.

Field names versus business meaning

- Match field names exactly unless the JSON contract explicitly maps them with an annotation.

- Check capitalization such as creationAt, updatedAt, firstName and lastName.
- A Java field may have a different name if it is mapped intentionally with @JsonProperty.
- Do not add fields merely because they appear in a response. Confirm whether the create endpoint accepts them.

12. Source-code issues and corrections

These are visible problems in the uploaded lesson examples. Fix these before using the snippets as runnable code.

Source issue	Why it fails or is risky	Correction
mapper.writeValueAsString(root PojoClassObjectName)	Placeholder variable is undefined.	Use the actual variable name, e.g. mapper.writeValueAsString(user).
address.setStreet(""GM Road")	Invalid Java string quotation.	Use address.setStreet("GM Road");
private Stritng slug;	Stritng is misspelled.	Use private String slug;
Missing semicolon after List.of(...)	Java statement will not compile.	End the declaration with);
List<Avater> listAvater	Type spelling does not match Avatar class.	Use List<Avatar> avatars.
Use of "25.45.45" as lng	Not a conventional decimal coordinate.	Use a valid value that matches the API schema.
response variable spelled repsonse	Identifier does not match declared variable.	Use response.then()...
request.log().all() on every test	Can leak PII, tokens or passwords.	Log safe details; use conditional logs in tests.

13. Best practices for POJO-based API automation

Model the contract, not an imagined payload

- Read Swagger/OpenAPI, API documentation or a real sample response/request.
- Separate request and response DTOs when their field sets differ; a response may include id/timestamps that should not be sent in a create request.
- Check required/optional fields and exact JSON types. A string "23999" is not the same as number 23999.
- Use consistent Java class names such as ProductRequest, ProductResponse, Category and Avatar.

Choose data types carefully

Field kind	Recommended consideration
IDs/counts	Use Integer/Long when null is meaningful; primitive int/long defaults to zero.
Money/prices	Prefer BigDecimal for monetary values when decimals and exact precision matter.
True/false flags	Boolean allows null; boolean only represents true/false.
Phone numbers/ZIP codes	Usually String because they may contain leading zeros, plus signs or formatting.
Date/time	Use documented ISO-8601 strings for simple examples; for production models, use suitable Java time types and configure Jackson Java Time support.
Optional fields	Wrapper types help distinguish absent/null from a default primitive value, but test actual serialization behavior.

Secure logging and test data

- Avoid logging complete requests where they contain passwords, tokens, addresses, phone numbers or other personal data.
- Do not use @Data on secrets without considering generated toString() output.
- Use non-real test identities and reserved/test domains where supported; do not post real personal information into public practice APIs.
- Keep base URLs and credentials in configuration/environment variables, not source code.

Assertions matter

- Do not stop at response.then().log().all(); logging is not validation.
- Assert exact status codes and required business fields according to the agreed API contract.
- Validate nested properties and list size/content for complex requests.
- For create operations, capture the returned ID only after asserting that the create request succeeded.

Keep the framework maintainable

- Use @Test methods rather than only main() methods for automated suites.
- Create reusable request specifications for shared base URI, headers and logging rules.
- Use unique test data to avoid duplicate-email or duplicate-slug conflicts.
- Clean up created records where the API permits it; avoid tests depending on stale records.

14. Common errors and troubleshooting

Symptom	Likely cause	What to check
415 Unsupported Media Type	Content-Type missing/wrong.	Set contentType("application/json") for JSON payloads.
400 Bad Request	Incorrect schema, wrong type or missing field.	Compare serialized JSON with the contract and inspect validation error.
Fields serialize as null/missing	Getter/field mapping or mapper configuration issue.	Check Lombok annotation processing, getters, JsonProperty and ObjectMapper settings.
Nested address missing	Parent field not assigned.	Confirm user.setAddress(address) was called.
Empty images/avatar array	List not populated or assigned.	Inspect list creation and products.setImages(images)/users.setAvatar(avatars).
Compilation errors	Typos, missing imports/semicolons or package mismatch.	Check String spelling, List imports, exact class names and ObjectMapper exception handling.
Public API returns unexpected result	Mock behavior or contract may differ.	Inspect actual response; do not assume a practice endpoint persists posted data.
Test passes without meaningful proof	Only response log is present.	Add status, content type and key field assertions.

15. More realistic test scenarios

Scenario A — Create a user profile

Requirement: user creation accepts name, contact details, address with location, and company details. Verify values survive serialization and come back in the response.

```
response.then()
   .statusCode(201) // use the API contract's exact expected code
   .body("name", equalTo("Mayuri"))
   .body("address.street", equalTo("GM Road"))
   .body("address.geo.lat", equalTo("24.455"))
   .body("company.name", equalTo("Amdocs"));
```

Scenario B — Create an e-commerce product

Requirement: product has a positive price, a category, and at least one image.

```
response.then()
   .statusCode(201)
   .body("price", greaterThan(0))
   .body("category.name", not(isEmptyOrNullString()))
   .body("images.size()", greaterThanOrEqualTo(1));
```

The sample matcher imports must be present for these assertions. Matchers should reflect the API contract and Java/Hamcrest version in your project.

Scenario C — Update profile skills and avatars

- Send an empty skills array and observe whether it is valid or rejected.
- Send duplicate skill values and verify documented behavior.
- Send an avatar with percent outside the accepted range and expect a validation error if the API defines that limit.
- Send avatar.size() as a negative number in a negative test; do not assume the service validates it unless the contract says so.

16. Hands-on exercises

Try each task before reading the solution outline. Use a local test API or a permitted practice endpoint and adjust expected responses to its actual contract.

1. Create a Geo POJO with lat and lng, using private fields and getters/setters.
2. Create Address with street, suite, city, zipcode and Geo geo. Populate it with a Geo object.
3. Create a Users object that includes nested Address and Company objects.
4. Serialize a Users object using ObjectMapper.writeValueAsString and print the JSON.
5. Send the same object with request.body(users) and compare the output JSON to the explicit mapper approach.
6. Create a Products class with title, price, Category and List<String> images.
7. Add three image URLs and assert the response images array has three elements.
8. Create a List<Avatar> with two objects and assign it to a Users payload.
9. Add @JsonProperty("long") to map Java field geoLong to JSON property long.
10. Fix the source typo Stritng and missing semicolon errors.
11. Write a TestNG test that verifies status code and two nested response values.

12. Review logging in your code and remove any password/token or unnecessary personal data from logs.

17. Solution patterns and worked examples

Solution pattern: assert list size and an element

```
response.then()
   .body("images.size()", equalTo(3))
   .body("images", hasItem("https://example.test/a2.png"));
```

Solution pattern: verify nested data

```
response.then()
   .body("address.city", equalTo("Pune"))
   .body("address.geo.lat", equalTo("24.455"))
   .body("company.name", equalTo("Amdocs"));
```

Solution pattern: log only on failure

```
response.then()
   .log().ifValidationFails()
   .statusCode(201)
   .body("title", equalTo("Samsung 10% Offer"));
```

If your framework version does not support a particular logging convenience method, log the response in a controlled failure handler instead. Never include secrets in the output.

Mini capstone — Product request test

Implement a TestNG test that creates a ProductRequest model, attaches a Category and a list of three image URLs, posts to a configured base URI, and asserts: expected status, non-empty returned identifier (if specified), title, category name, positive price, and image count. Keep expected response details aligned with the real endpoint's contract.

18. Interview revision questions

What is a POJO?

A normal Java class used to represent request/response data.

What is serialization?

Converting a Java object into JSON or another transport representation.

Why use nested POJOs?

They model nested JSON objects using the same hierarchy.

What does List<String> become in JSON?

An array of strings.

What does List<Avatar> become in JSON?

An array whose elements are objects with Avatar fields.

What is ObjectMapper?

Jackson's utility for converting between Java objects and JSON.

Difference between request.body(user) and mapper.writeValueAsString(user)?

The first lets Rest Assured serialize the object; the second creates JSON text explicitly before attaching it to the request.

What does @Data do?

Lombok generates getters, setters, toString, equals and hashCode.

Why should secrets not be logged?

Logs are often retained and shared; tokens/passwords can be used to impersonate users.

Why separate request and response DTOs?

They frequently have different fields, validation rules and lifecycle semantics.

19. Final checklist before submitting a POJO test

- ☐ Class fields and names match the request JSON contract.
- ☐ Nested properties are represented by nested POJO fields.
- ☐ Arrays/lists use the correct element type.
- ☐ Each nested object is created and assigned to its parent.
- ☐ Content-Type is application/json for JSON request bodies.
- ☐ Jackson/Lombok dependencies and IDE annotation processing are configured.
- ☐ Serialization output has been inspected during local debugging.
- ☐ Assertions verify the API contract, not only that a response exists.
- ☐ Generated IDs are captured only after verifying successful creation.
- ☐ No secrets or real personal information are exposed in logs.
- ☐ Test data is unique enough to avoid conflicts and cleanup is considered.

Closing summary

The most important skill in this class is learning to translate a JSON contract into a matching Java object graph. A field holding a nested JSON object becomes a field of another POJO type; a JSON array becomes a Java List; and the top-level request object is serialized to JSON using Rest Assured/Jackson. Once you can model the structure accurately, constructing request data and validating nested responses become much easier.

Keep this sequence in mind Read the API contract → create POJO classes → populate nested objects/lists → serialize → send request → assert response → inspect failures → improve the model.